

Automatic Bearer Token Injection Using a Custom DelegatingHandler

Eliminate repetitive JWT header code from every component
with a centralised, reusable HTTP pipeline handler.

Technology: .NET 10 · Blazor WebAssembly · ASP.NET Core Identity · JWT

1 · Problem Statement

In the current **EmployeeApp.UI** project, every Blazor component that needs to call a secured API endpoint copies the same three lines before the request:

 EMPLOYEELIST.RAZOR (CURRENT PATTERN)

```
var token = await Js.InvokeAsync<string>("localStorage.getItem",  
    "token");
```

```
Http.DefaultRequestHeaders.Authorization =  
    new AuthenticationHeaderValue("Bearer", token);
```

```
Employees = await Http.GetFromJsonAsync<List<EmployeeDto>>  
    ("api/employees");
```

Problem	Why it matters
Code duplication	The same 3 lines appear in <i>every</i> component that calls the API — brittle and hard to maintain.
Shared mutable state	<code>DefaultRequestHeaders</code> is shared across all requests on the same <code>HttpClient</code> instance. Mutating it from one component can interfere with concurrent requests.

Wrong**header target**

`DefaultRequestHeaders` applies to the client object. The correct place to set an Authorization header is `request.Headers` (per-request), which is thread-safe.

Leaky**abstraction**

UI components must know about token storage (`localStorage`). They should only care about data, not authentication mechanics.

2 · The Solution — Custom `DelegatingHandler`

A **`DelegatingHandler`** is a piece of middleware that sits inside the `HttpClient` request pipeline. Every time *any* component sends an HTTP request through the client, the handler runs first — it reads the JWT from `localStorage` and attaches it to the request header automatically. Components never need to touch headers again.

```
Component // calls Http.GetFromJsonAsync(), Http.DeleteAsync(), etc. ↓
AuthTokenHandler.SendAsync() // ← our new handler reads token from
localStorage sets request.Headers.Authorization = "Bearer <token>" ↓
HttpClientHandler // built-in handler – sends the request over the wire ↓
ASP.NET Core API // receives request with Authorization header already
set
```

✅ Key Benefit

The `AuthTokenHandler` is registered *once* in `Program.cs`. Every component benefits automatically — zero per-component changes needed.

3 · Step-by-Step Implementation

1 Add `GetTokenAsync()` to `CustomAuthenticationStateProvider`

The handler needs a way to retrieve the current JWT. Rather than letting it call JavaScript directly (which would couple it to `localStorage`), we expose a single helper method on the existing `CustomAuthenticationStateProvider`.

 AUTH/CUSTOMAUTHENTICATIONSTATEPROVIDER.CS — ADD THIS METHOD

```
// Returns the raw JWT string stored in the browser's localStorage,  
// or null if the user is not logged in.  
public async Task<string?> GetTokenAsync()  
{  
    return await _js.InvokeAsync<string>("localStorage.getItem",  
    "token");  
}
```

- ① `Task<string?>` — the method is asynchronous because calling JavaScript via `IJSRuntime` in Blazor WASM is an async operation (it crosses the .NET ↔ JS boundary).
- ② `_js.InvokeAsync<string>` — calls the browser's native `localStorage.getItem("token")` function and returns the stored string value. If the key does not exist, it returns `null`.
- ③ `string?` nullable return — if the user is not logged in, the token will be `null`. The handler checks for this before setting the header.

2 Create the `AuthTokenHandler` class

Create a new file `Auth/AuthTokenHandler.cs`. This class extends `DelegatingHandler`, which is the standard .NET base class for HTTP middleware handlers.

 AUTH/AUTHTOKENHANDLER.CS — CREATE THIS NEW FILE

```
using System.Net.Http.Headers;  
  
namespace EmployeeApp.UI.Auth;  
  
public class AuthTokenHandler : DelegatingHandler // ①  
{  
    private readonly CustomAuthenticationStateProvider _authProvider;  
    // ②  
  
    public AuthTokenHandler(CustomAuthenticationStateProvider
```

```

authProvider) // ③
{
    _authProvider = authProvider;
}

protected override async Task<HttpResponseMessage> SendAsync(
// ④
    HttpRequestMessage request,
    CancellationToken cancellationToken)
{
    var token = await _authProvider.GetTokenAsync();
// ⑤

    if (!string.IsNullOrEmpty(token))
// ⑥
        request.Headers.Authorization =
            new AuthenticationHeaderValue("Bearer", token); // ⑦

    return await base.SendAsync(request, cancellationToken); //
// ⑧
}
}

```

- ① **: DelegatingHandler** — inheriting from this base class lets us hook into the **HttpClient** pipeline. The base class handles the chain of responsibility: it calls the next handler after ours finishes.
- ② We store a reference to **CustomAuthenticationStateProvider** so we can call its **GetTokenAsync()** method. This is plain **constructor injection**.
- ③ The constructor accepts the provider. The ASP.NET Core DI container automatically supplies it when the handler is resolved — no manual wiring needed.
- ④ **SendAsync** is the single method we override. It is called by the runtime for every HTTP request made through the associated **HttpClient**. The **request** parameter represents the outgoing HTTP request we can modify.
- ⑤ We read the JWT from localStorage via the provider. This is the *only* place in the entire app where this read happens for outgoing API calls.
- ⑥ We only set the header when a token actually exists. If the user is not logged in, the request goes through unauthenticated — which is correct for public endpoints.
- ⑦ **request.Headers.Authorization** — this is the **per-request** header, not the shared **DefaultRequestHeaders**. It is set fresh for every individual request, making it completely thread-safe.
- ⑧ **base.SendAsync(...)** — passes the (now-modified) request to the *next* handler in the pipeline (the built-in **HttpClientHandler** that performs the actual network call) and returns its response back up the chain.

3 Register everything in **Program.cs**

Replace the current single-line `HttpClient` registration with a named client that has the handler attached, then keep a convenience scoped alias so all existing `@inject HttpClient Http` lines continue to work unchanged.

PROGRAM.CS — REPLACE THE HTTPCLIENT REGISTRATION

```
// — Before (remove this line)

// builder.Services.AddScoped(sp =>
//     new HttpClient { BaseAddress = new
// Uri("https://localhost:7021/") });

// — After (add these lines)

builder.Services.AddTransient<AuthTokenHandler>(); // ①

builder.Services
// ②
    .AddHttpClient("Api", client =>
        client.BaseAddress = new Uri("https://localhost:7021/")) //
// ③
    .AddHttpMessageHandler<AuthTokenHandler>();
// ④

builder.Services.AddScoped(sp => //
// ⑤
    sp.GetRequiredService<IHttpClientFactory>()
        .CreateClient("Api"));
```

- ① `AddTransient<AuthTokenHandler>()` — registers the handler with a **Transient** lifetime. DelegatingHandlers must be Transient (not Scoped) to avoid socket exhaustion; the factory manages the actual handler instance reuse internally.
- ② `AddHttpClient("Api", ...)` — creates a **named HttpClient** configuration called `"Api"`. Named clients are managed by `IHttpClientFactory`, which correctly pools the underlying `HttpClientHandler` to avoid socket exhaustion.
- ③ Sets the `BaseAddress` on the named client exactly as before, so relative URLs like `"api/employees"` still resolve correctly.
- ④ `AddHttpMessageHandler<AuthTokenHandler>()` — inserts our handler into the pipeline for this named client. This is the single line that makes the automatic token injection work for every request.
- ⑤ Registers a **Scoped HttpClient** by resolving the named client from the factory. This means `@inject HttpClient Http` in all existing Blazor components continues to work without any changes to those files.

4 Clean up the components

Now that the handler manages the token header, remove the duplicated token code from every component.

X BEFORE (REMOVE ALL OF THIS)

```
@using System.Net.Http.Headers
@Inject IJSRuntime Js

// In every method:
var token = await Js
    .InvokeAsync<string>(
        "localStorage.getItem",
        "token");

Http.DefaultRequestHeaders
    .Authorization =
        new AuthenticationHeaderValue(
            "Bearer", token);
```

✓ AFTER (JUST MAKE THE CALL)

```
// No @using, no @inject
IJSRuntime

// No token reading
// No header setting

Employees = await Http
    .GetFromJsonAsync<
        List<EmployeeDto>>(
            "api/employees");

// Header is already set
// by AuthTokenHandler ✓
```

4 · Complete File Reference

Final CustomAuthenticationStateProvider.cs (only the addition)

 AUTH/CUSTOMAUTHENTICATIONSTATEPROVIDER.CS - ONLY SHOWING THE NEW METHOD

```
// Existing class body stays unchanged.
// Add only this one method:

public async Task<string?> GetTokenAsync()
{
    return await _js.InvokeAsync<string>("localStorage.getItem",
    "token");
}
```

Final AuthTokenHandler.cs (complete file)

 AUTH/AUTHTOKENHANDLER.CS

```
using System.Net.Http.Headers;

namespace EmployeeApp.UI.Auth;

public class AuthTokenHandler : DelegatingHandler
{
    private readonly CustomAuthenticationStateProvider _authProvider;

    public AuthTokenHandler(CustomAuthenticationStateProvider
authProvider)
    {
        _authProvider = authProvider;
    }

    protected override async Task<HttpResponseMessage> SendAsync(
        HttpRequestMessage request,
        CancellationToken cancellationToken)
    {
        var token = await _authProvider.GetTokenAsync();

        if (!string.IsNullOrEmpty(token))
            request.Headers.Authorization =
                new AuthenticationHeaderValue("Bearer", token);
    }
}
```

```
        return await base.SendAsync(request, cancellationToken);
    }
}
```

Final Program.cs (complete file)

PROGRAM.CS

```
using EmployeeApp.UI;
using EmployeeApp.UI.Auth;
using Microsoft.AspNetCore.Components.Authorization;
using Microsoft.AspNetCore.Components.Web;
using Microsoft.AspNetCore.Components.WebAssembly.Hosting;

var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("head::after");

builder.Services.AddTransient<AuthTokenHandler>();

builder.Services
    .AddHttpClient("Api", client =>
        client.BaseAddress = new Uri("https://localhost:7021/"))
    .AddHttpMessageHandler<AuthTokenHandler>();

builder.Services.AddScoped(sp =>
    sp.GetRequiredService<IHttpClientFactory>()
        .CreateClient("Api"));

builder.Services.AddAuthorizationCore();
builder.Services.AddScoped<CustomAuthenticationStateProvider>();
builder.Services.AddScoped<AuthenticationStateProvider>(sp =>
    sp.GetRequiredService<CustomAuthenticationStateProvider>());

await builder.Build().RunAsync();
```

5 · Implementation Checklist for Trainees

#	Task	File
1	Add <code>GetTokenAsync()</code> method	<code>Auth/CustomAuthenticationStateProvider.cs</code>
2	Create the <code>AuthTokenHandler</code> class	<code>Auth/AuthTokenHandler.cs</code> (new file)
3	Register handler and replace <code>HttpClient</code> registration	<code>Program.cs</code>
4	Remove token-reading code from each component	All <code>.razor</code> files that called the API
5	Remove unused <code>@using</code> and <code>@inject IJSRuntime Js</code> from those components	All <code>.razor</code> files cleaned in step 4
6	Run the app and verify login → employee list works	—

6 · Common Mistakes to Avoid

⚠ Registering the handler as Scoped instead of Transient

`DelegatingHandler` must be registered as `AddTransient`. Registering as `AddScoped` can cause socket exhaustion because the handler's inner handler may be disposed prematurely.

⚠ Forgetting the Scoped alias for `HttpClient`

If you only call `AddHttpClient("Api", ...)` and forget the `AddScoped(sp => factory.CreateClient("Api"))` line, `@inject HttpClient Http` in your Razor components will fail at runtime with a DI resolution error.

⚠ Still using `DefaultRequestHeaders`

After adding the handler, remove *all* remaining `Http.DefaultRequestHeaders.Authorization = ...` lines. If both exist, the component line will overwrite the handler's header and you may end up with the wrong (stale) token.

💡 Tip — Public endpoints

The handler only sets the header when a token exists. If a user is not logged in, requests go through without an Authorization header, which is the correct behaviour for public endpoints like `POST /api/auth/login`.